LESSON 1

Digital and Analogue Signals

Learning outcomes. By the end of this lesson you should be able to:

- define a **binary value** and give its two possible states;
- describe how binary values are represented by **voltage levels**;
- explain why digital systems are generally more reliable than analogue ones.

1.1 Binary values

A **binary value** is a piece of information that can take only one of two possible states, usually written as 0 or 1. In computing, 0 often means "off" and 1 means "on". These two symbols are the building blocks of all digital data.

Worked example

A simple counter that counts from 0 to 3 uses two binary values: the first bit b_1 and the second bit b_0. The number 2 is represented as $b_1 b_0 = 10$, meaning $b_1 = 1$ and $b_0 = 0$.

1.2 Voltage levels

Digital circuits map binary values to specific voltage ranges. A common convention is:

$$\text{Low voltage (e.g., } 0 \text{ V to } 0.8 \text{ V)} \to 0, \qquad \text{High voltage (e.g., } 2 \text{ V to } 5 \text{ V)} \to 1.$$

These ranges are called **logic levels**. The gap between the high and low ranges provides noise margin: small variations in voltage do not change the interpreted binary value.

Worked example

In a TTL (Transistor-Transistor Logic) gate, a voltage of $0.2\,\text{V}$ is read as 0, while $3.5\,\text{V}$ is read as 1. If a signal drifts to $1.0\,\text{V}$, it is still within the low range and will be interpreted as 0.

1.3 Why digital systems are reliable

Digital systems are reliable for three main reasons:

- **Noise immunity.** Because only two voltage bands are used, small electrical noise rarely pushes a signal across the threshold.
- **Regeneration.** Digital repeaters restore signals to exact logic levels, eliminating accumulated distortion.
- **Error detection.** Techniques such as parity bits or checksums can detect and sometimes correct errors in transmitted binary data.

Worked example

A digital communication link sends the byte 10101100. A parity bit is added so the total number of 1s is even. If a single bit flips during transmission, the receiver detects the mismatch and can request a retransmission.

Key test. Identify a device that uses voltage levels to represent 0 and 1. Explain how the device would behave if a small amount of electrical noise is added to the signal.

1.4 Exercises

1. Write the decimal numbers 0 to 3 in binary using two bits.
2. Sketch the voltage ranges for logic 0 and logic 1 in a CMOS circuit.
3. List two ways digital systems cope with noise, and give a brief example of each.

1.5 Key terms

Term	Meaning
Binary value	A datum that can be either 0 or 1.
Voltage level	The range of electrical potential that represents a binary 0 (low) or 1 (high).
Noise immunity	The ability of a system to tolerate small unwanted variations without misinterpreting data.
Regeneration	The process of restoring a digital signal to its ideal logic levels.
Error detection	Methods such as parity bits that reveal when transmitted data has been altered.

LESSON 2

Number Systems and Codes

Learning outcomes. By the end of this lesson you should be able to:

- write numbers in binary, octal and hexadecimal forms;
- convert between decimal, binary, octal and hexadecimal;
- explain the purpose of binary-coded decimal (BCD);
- perform simple BCD conversions.

2.1 Binary system

The **binary** system uses only two digits, 0 and 1. Each position represents a power of 2, starting with 2^0 at the right-most digit.

For example, the binary number 1011_2 equals:

$$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11_{10}.$$

Worked example

Convert 11010_2 to decimal.

$$1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 16 + 8 + 0 + 2 + 0 = 26_{10}.$$

2.2 Octal system

The **octal** system uses digits 0–7. Each position represents a power of 8.

For instance, 157_8 equals:

$$1 \cdot 8^2 + 5 \cdot 8^1 + 7 \cdot 8^0 = 64 + 40 + 7 = 111_{10}.$$

Worked example

Convert 73_8 to decimal.

$$7 \cdot 8^1 + 3 \cdot 8^0 = 56 + 3 = 59_{10}.$$

2.3 Hexadecimal system

The **hexadecimal** system uses sixteen symbols: 0–9 and A–F (where A=10, …, F=15). Each position represents a power of 16.

Example: $2B3_{16}$ equals:

$$2 \cdot 16^2 + 11 \cdot 16^1 + 3 \cdot 16^0 = 512 + 176 + 3 = 691_{10}.$$

Worked example

Convert $1F_{16}$ to decimal.

$$1 \cdot 16^1 + 15 \cdot 16^0 = 16 + 15 = 31_{10}.$$

2.4 Conversions between bases

2.4.1 Decimal to binary

Repeatedly divide by 2, recording remainders. The binary digits are the remainders read upward.

Worked example

Convert 45_{10} to binary.

$$45/2 = 22 \text{ r } 1,\ 22/2 = 11 \text{ r } 0,\ 11/2 = 5 \text{ r } 1,\ 5/2 = 2 \text{ r } 1,\ 2/2 = 1 \text{ r } 0,\ 1/2 = 0 \text{ r } 1.$$

Reading remainders upward gives 101101_2.

2.4.2 Binary to octal

Group binary digits in threes from the right; replace each group by its octal value.

Worked example

Convert 1101101_2 to octal.

$$110\ 110\ \underline{1000} \to 6\ 6\ 0_8 = 660_8.$$

(Added leading zeros to make a full group of three.)

2.4.3 Binary to hexadecimal

Group binary digits in fours from the right; replace each group by its hexadecimal value.

Worked example

Convert 10111011_2 to hexadecimal.

$$1011\ 1011 \to B\ B_{16} = BB_{16}.$$

2.4.4 Octal to binary and hexadecimal

Convert each octal digit to a three-bit binary group, then apply the binary-to-hex method if needed.

Worked example

Convert 57_8 to hexadecimal.

$$5_8 = 101_2,\ 7_8 = 111_2 \ \Rightarrow\ 101111_2.$$

Group as $0010\ 1111 \to 2F_{16}$.

2.5 Binary-coded decimal (BCD)

Binary-coded decimal stores each decimal digit separately as a four-bit binary number. For example, the decimal number 93 is represented as $1001\ 0011_{BCD}$ (9 $\to$ 1001, 3 $\to$ 0011).

BCD is useful in digital displays because each digit can be driven independently.

Worked example

Encode 254 in BCD.

$$2 \to 0010,\ 5 \to 0101,\ 4 \to 0100 \ \Rightarrow\ 0010\ 0101\ 0100_{BCD}.$$

Key test. Convert 101101_2 to octal, hexadecimal and decimal. Then write the decimal result in BCD.

2.6 Exercises

1. Write 27_{10} in binary, octal and hexadecimal.
2. Convert $3F_{16}$ to binary and then to octal.
3. Encode the decimal number 718 in BCD.
4. Decode the BCD value $0100\ 1001\ 0010_{BCD}$ to a decimal number.

2.7 Key terms

Term	Meaning
Binary	Base-2 numeral system using digits 0 and 1.
Octal	Base-8 numeral system using digits 0–7.
Hexadecimal	Base-16 numeral system using digits 0–9 and A–F.
Conversion	Process of changing a number from one base to another.
Binary-coded decimal (BCD)	Representation of each decimal digit by a separate 4-bit binary code.

LESSON 3

Boolean Algebra

Learning outcomes. By the end of this lesson you should be able to:

- state the basic Boolean laws;
- construct truth tables for Boolean expressions;
- apply the laws to simplify Boolean expressions.

3.1 Boolean laws

The following identities hold for any Boolean variables A, B, C.

- **Identity laws**: $A + 0 = A$, $A \cdot 1 = A$.
- **Null laws**: $A + 1 = 1$, $A \cdot 0 = 0$.
- **Idempotent laws**: $A + A = A$, $A \cdot A = A$.
- **Complement laws**: $A + \overline{A} = 1$, $A \cdot \overline{A} = 0$.
- **Commutative laws**: $A + B = B + A$, $A \cdot B = B \cdot A$.
- **Associative laws**: $(A + B) + C = A + (B + C)$, $(A \cdot B) \cdot C = A \cdot (B \cdot C)$.
- **Distributive laws**: $A \cdot (B + C) = A \cdot B + A \cdot C$, $A + (B \cdot C) = (A + B) \cdot (A + C)$.
- **De Morgan's laws**: $\overline{A + B} = \overline{A} \cdot \overline{B}$, $\overline{A \cdot B} = \overline{A} + \overline{B}$.

Worked example

Simplify $A \cdot 1 + A \cdot \overline{A}$.

$$A \cdot 1 + A \cdot \overline{A} = A + A \cdot \overline{A} = A(1 + \overline{A}) = A \cdot 1 = A.$$

The identity law $A \cdot 1 = A$ and the complement law $1 + \overline{A} = 1$ were used.

3.2 Truth tables

A truth table lists the output of a Boolean expression for every combination of its input variables.

Worked example

Truth table for $F = A + \overline{B}$.

A	B	F
0	0	$0 + 1 = 1$
0	1	$0 + 0 = 0$
1	0	$1 + 1 = 1$
1	1	$1 + 0 = 1$

3.3 Simplifying expressions

To simplify a Boolean expression, repeatedly apply the Boolean laws until no further reduction is possible.

Worked example

Simplify $F = \overline{A \cdot B} + A \cdot B$.

$$\begin{aligned} F &= \overline{A \cdot B} + A \cdot B \\ &= (\overline{A} + \overline{B}) + A \cdot B \quad \text{(De Morgan)} \\ &= (\overline{A} + \overline{B} + A) \cdot (\overline{A} + \overline{B} + B) \quad \text{(Distributive)} \\ &= (1 + \overline{B}) \cdot (\overline{A} + 1) \quad \text{(Complement, Identity)} \\ &= 1 \cdot 1 = 1. \end{aligned}$$

Thus F simplifies to the constant 1.

Key test. Reduce the expression $G = (A + B) \cdot (\overline{A} + B) \cdot (A + \overline{B})$ using the Boolean laws. Show each step.

3.4 Exercises

1. Write the truth table for $H = \overline{A} \cdot B + A \cdot \overline{B}$.
2. Using the Boolean laws, simplify $J = (A + 0) \cdot (B + 1)$.
3. Prove De Morgan's first law $\overline{A + B} = \overline{A} \cdot \overline{B}$ by constructing a truth table.

3.5 Key terms

Term	Meaning
Boolean law	An identity that holds for all Boolean variables (e.g., identity, complement).
Truth table	A tabular method that lists the output of a Boolean expression for every input combination.
Simplification	The process of reducing a Boolean expression to a shorter equivalent form using the laws.
De Morgan's laws	Two transformation rules that relate complements of sums and products.

LESSON 4

Logic Gates

Learning outcomes. By the end of this lesson you should be able to:

- describe the function of the basic logic gates AND, OR, NOT, NAND, NOR and XOR;
- draw the symbol for each gate;
- write the truth table for each gate;
- combine gates to implement simple Boolean expressions.

4.1 AND gate

The **AND** gate outputs 1 only when all its inputs are 1. Its symbol is a flat-sided rectangle with a curved back.

$$\text{output} = A \wedge B$$

Worked example

For inputs $A = 1$, $B = 0$ the AND gate gives $1 \wedge 0 = 0$. The output is 0.

4.2 OR gate

The **OR** gate outputs 1 when at least one input is 1. Its symbol is a curved shape pointing to the right.

$$\text{output} = A \vee B$$

Worked example

With $A = 0$, $B = 1$ the OR gate gives $0 \vee 1 = 1$. The output is 1.

4.3 NOT gate

The **NOT** gate (inverter) has one input and produces the opposite logical value.

$$\text{output} = \neg A$$

Worked example

If $A = 0$ then $\neg A = 1$. The output is 1.

4.4 NAND gate

The **NAND** gate is the negation of AND. It outputs 0 only when all inputs are 1.

$$\text{output} = \neg(A \wedge B)$$

Worked example

For $A = 1$, $B = 1$ the NAND gate gives $\neg(1 \wedge 1) = \neg 1 = 0$. The output is 0.

4.5 NOR gate

The **NOR** gate is the negation of OR. It outputs 1 only when all inputs are 0.

$$\text{output} = \neg(A \vee B)$$

Worked example

With $A = 0$, $B = 0$ the NOR gate gives $\neg(0 \vee 0) = \neg 0 = 1$. The output is 1.

4.6 XOR gate

The **XOR** (exclusive OR) gate outputs 1 when exactly one input is 1.

$$\text{output} = A \oplus B$$

Worked example

If $A = 1$, $B = 0$ then $A \oplus B = 1$. The output is 1. If both are 1, the output is 0.

4.7 Building circuits from gates

Complex Boolean functions can be built by connecting basic gates. For example, the expression $F = (A \wedge B) \vee (\neg C)$ can be realised with one AND gate, one NOT gate, and one OR gate. The output of the AND gate and the NOT gate are fed into the OR gate.

Worked example

Implement $F = (A \wedge B) \vee (\neg C)$. 1. Connect A and B to an AND gate. 2. Connect C to a NOT gate. 3. Connect the outputs of the AND and NOT gates to an OR gate. The final output of the OR gate is F.

Key test. Draw the circuit that implements $G = \neg(A \vee B) \wedge (C \oplus D)$. Identify each gate used.

4.8 Exercises

1. Write the truth table for a two-input NAND gate.
2. Simplify the expression $H = \neg(A \wedge B) \vee (A \wedge \neg B)$ using gate symbols.
3. Design a circuit that outputs 1 only when exactly two of the three inputs X, Y, Z are 1. Indicate the gates required.

4.9 Key terms

Term	Meaning
AND	Outputs 1 only if all inputs are 1.
OR	Outputs 1 if any input is 1.
NOT	Inverts a single input.
NAND	Negated AND; outputs 0 only when all inputs are 1.
NOR	Negated OR; outputs 1 only when all inputs are 0.
XOR	Outputs 1 when an odd number of inputs are 1.
Circuit	Arrangement of gates that implements a Boolean function.

LESSON 5

Combinational Circuits

Learning outcomes. By the end of this lesson you should be able to:

- describe the operation of a half-adder and a full-adder;
- construct a 2-to-1 multiplexer and explain its truth table;
- design a binary decoder and state its applications;
- explain how an encoder performs the inverse mapping of a decoder;
- analyse simple combinational networks for correct output.

5.1 Adders

A **half-adder** adds two single binary digits A and B. It produces a *sum* S and a *carry* C. The logic equations are $S = A \oplus B$ and $C = A \cdot B$.

A **full-adder** adds three bits: two data bits A, B and an incoming carry C_{in}. It yields a sum S and a carry out C_{out} with

$$S = A \oplus B \oplus C_{\text{in}}, \qquad C_{\text{out}} = (A \cdot B) + (B \cdot C_{\text{in}}) + (A \cdot C_{\text{in}}).$$

Worked example

Add $A = 1$, $B = 1$, $C_{\text{in}} = 0$ using a full-adder. $S = 1 \oplus 1 \oplus 0 = 0$, $C_{\text{out}} = (1{\cdot}1) + (1{\cdot}0) + (1{\cdot}0) = 1$. Result is binary 10, i.e., 2 in decimal.

5.2 Multiplexers

A **multiplexer** (or **mux**) selects one of several input lines and forwards it to a single output line, based on selector bits. A 2-to-1 mux has inputs D_0, D_1, selector S, and output Y with

$$Y = \overline{S}\, D_0 + S\, D_1.$$

Worked example

Let $D_0 = 0$, $D_1 = 1$, and selector $S = 1$. Then $Y = \overline{1} \cdot 0 + 1 \cdot 1 = 1$. The mux passes D_1 to the output.

5.3 Decoders

A **decoder** converts a binary code into a one-hot output. A 2-to-4 decoder has two selector inputs $A_1 A_0$ and four outputs Y_0–Y_3, where $Y_i = 1$ only when the binary input equals i.

Worked example

For input $A_1 A_0 = 10$ (binary 2), the decoder sets $Y_2 = 1$ and all other $Y_i = 0$.

5.4 Encoders

An **encoder** performs the opposite mapping of a decoder: it receives a one-hot input and produces the binary code of the active line. A 4-to-2 encoder has inputs $D_3 - D_0$ and outputs $A_1 A_0$, where $A_1 A_0$ is the binary index of the high input.

Worked example

If $D_2 = 1$ and all other $D_i = 0$, the encoder outputs $A_1A_0 = 10$ (binary 2).

Key test. Design a circuit that adds two 2-bit numbers using full-adders, and verify its output for the input pair $01 + 11$.

5.5 Exercises

1. Draw the truth table for a half-adder and a full-adder.
2. Implement a 2-to-1 mux using only NAND gates.
3. Write the Boolean expressions for the outputs of a 3-to-8 decoder.
4. Given a one-hot input $D_0 = 0$, $D_1 = 0$, $D_2 = 1$, $D_3 = 0$, state the encoder output.
5. Combine two full-adders to create a 2-bit adder and compute the sum of 10_2 and 01_2.

5.6 Key terms

Term	Meaning
Half-adder	Circuit that adds two single bits, producing sum and carry.
Full-adder	Circuit that adds three bits (two data bits and a carry-in).
Multiplexer	Device that selects one of many inputs based on selector bits.
Decoder	Circuit that converts binary input to a one-hot output pattern.
Encoder	Circuit that converts a one-hot input to its binary index.

LESSON 6

Karnaugh Maps

Learning outcomes. By the end of this lesson you should be able to:

- identify how cells are grouped on a Karnaugh map;
- derive a minimal sum-of-products expression from a Karnaugh map;
- recognise the importance of adjacency and wrap-around.

6.1 Grouping cells

A **group** (or **implicant**) is a rectangle of 1, 2, 4, 8 ... cells that are adjacent horizontally or vertically. Cells on the left edge are adjacent to cells on the right edge, and cells on the top edge are adjacent to cells on the bottom edge (wrap-around). Each group must contain only 1s (or don't-cares) and must be as large as possible.

The number of variables eliminated by a group equals $\log_2$(number of cells in the group). For example, a group of four cells eliminates two variables, leaving a product term with the remaining variables.

Worked example

Consider the 3-variable map for $F(A, B, C)$ with 1s at minterms $m(0), m(1), m(2), m(3)$. These four cells form a 2×2 block. Because the block contains four cells, two variables are eliminated; the remaining variable is A', so the product term is A'.

6.2 Minimal sum-of-products expressions

To obtain a minimal sum-of-products (SOP) expression:

1. Plot all 1s (and optional don't-cares) on the Karnaugh map.
2. Form the largest possible groups covering every 1 at least once.
3. Write a product term for each group, omitting the variables that change within the group.
4. Combine the product terms with OR (+) to obtain the SOP.

If a 1 can be covered by more than one group, choose the covering that yields the fewest product terms or the simplest terms.

Worked example

For $F(A, B, C, D)$ the 1s occur at minterms $m(1), m(3), m(5), m(7), m(9), m(11), m(13), m(15)$. On a 4-variable map we can make two groups of four cells each:

- Group 1 covers $m(1), m(3), m(5), m(7) \rightarrow$ the term $B'D$ (since A and C vary).
- Group 2 covers $m(9), m(11), m(13), m(15) \rightarrow$ the term BD (since A and C vary).

The minimal SOP is $F = B'D + BD$ which simplifies further to $F = D(B' + B) = D$.

Key test. Given a Karnaugh map with 1s at $m(0), m(2), m(8), m(10)$ for a 4-variable function, draw the groups and write the minimal SOP expression.

6.3 Exercises

1. Plot the 1s for $F(A, B, C) = \sum m(0, 2, 3, 6, 7)$ on a 3-variable Karnaugh map and form the minimal SOP.

2. Explain why a group of eight cells in a 4-variable map eliminates three variables.

3. Using wrap-around, group the 1s at $m(1), m(3), m(9), m(11)$ in a 4-variable map and give the resulting product term.

6.4 Key terms

Term	Meaning
Group (implicant)	A rectangle of 2^n adjacent cells containing only 1s (or don't-cares); used to simplify Boolean expressions.
Wrap-around	The adjacency rule that cells on opposite edges of the map are considered neighbours.
Sum-of-products (SOP)	A Boolean expression formed by OR-ing several AND terms; each term corresponds to a group on the map.
Minimal SOP	The SOP expression with the fewest product terms and/or the fewest literals, obtained by using the largest possible groups.
Don't-care	A minterm that may be assigned either 0 or 1 to aid grouping; it is plotted as an X on the map.

LESSON 8

Counters and Registers

Learning outcomes. By the end of this lesson you should be able to:

- describe how a binary counter works;
- explain the function of a shift register;
- read and draw timing diagrams for counters and registers;
- design a simple synchronous binary counter using flip-flops.

7.1 Binary counters

A **binary counter** is a sequential circuit that counts upward in binary notation. Each clock pulse increments the count by one. The simplest form uses a chain of T-flip-flops (or JK-flip-flops with J=K=1). The output of one flip-flop feeds the clock input of the next, producing a ripple effect.

For a 3-bit counter the states progress as:

$$000 \to 001 \to 010 \to 011 \to 100 \to 101 \to 110 \to 111 \to 000$$

The least-significant bit (LSB) toggles on every clock pulse, the next bit toggles when the LSB changes from 1 to 0, and so on.

Worked example

A 2-bit binary counter built from two T-flip-flops has outputs Q_1 (MSB) and Q_0 (LSB). Starting at 00, after four clock pulses the sequence is $00, 01, 10, 11, 00$. The circuit diagram shows Q_0 connected to the clock of Q_1.

7.2 Shift registers

A **shift register** is a group of flip-flops connected in series so that data entered at one end moves one position per clock pulse. It can be used for temporary storage, data serialization, or delay lines.

In a 4-bit parallel-in parallel-out (PIPO) shift register, the parallel inputs $D_3 - D_0$ are loaded simultaneously when a load signal is high. When the load is low, each clock pulse shifts the bits right:

$$Q_3 \leftarrow Q_2,\ Q_2 \leftarrow Q_1,\ Q_1 \leftarrow Q_0,\ Q_0 \leftarrow \text{serial_in}$$

Worked example

Consider a 4-bit shift register with initial contents 1011. With serial input 0 and a load signal low, after three clock pulses the contents become 0101. The bits have moved right three positions, and the leftmost bits are filled with the serial input.

7.3 Timing diagrams

A **timing diagram** shows the logical levels of signals versus time. For counters and shift registers it displays the clock, control signals (load, enable), and the outputs Q_i.

Key features to read:

- Rising edge of the clock triggers state changes.
- When a control signal is active, the corresponding action (load or shift) occurs.

- Output waveforms follow the binary counting or shifting pattern.

Worked example

A timing diagram for a 3-bit binary counter shows the clock signal with regular rising edges. The Q_0 waveform toggles every edge, Q_1 toggles every second edge, and Q_2 toggles every fourth edge. The diagram also marks the count reset when a synchronous clear signal goes high.

Key test. Draw the timing diagram for a 2-bit binary counter over eight clock cycles, starting from 00. Indicate the clock, Q_1, and Q_0 waveforms.

7.4 Exercises

1. List the binary states of a 3-bit counter in order and indicate which flip-flop toggles on each clock pulse.

2. A 4-bit shift register receives the serial input sequence 1,0,1,0. Starting from 0000, write the register contents after each of the four clock pulses.

3. Explain why a ripple-counter may suffer from propagation delay, and how a synchronous counter avoids this problem.

7.5 Key terms

Term	Meaning
Binary counter	A sequential circuit that counts upward in binary with each clock pulse.
Shift register	A chain of flip-flops that moves data one position per clock pulse.
Timing diagram	Graphical representation of signal levels versus time.
Ripple counter	A counter where each flip-flop is clocked by the output of the previous stage, causing delay propagation.
Synchronous counter	A counter where all flip-flops receive the same clock edge, eliminating ripple delay.